

Semi-Secret Wars

AI Development Guide (ADG)

Version: 0.2

Status: Living Document

1. Purpose

This document defines the development workflow for Semi-Secret Wars — how the AI developer (Claude, working as Lead Gameplay Programmer) collaborates with the Designer using Godot 4.7-stable. The objective is small, testable milestones with high code quality and a scalable architecture. This defines **how** the game is developed, not how the game works — see the GDD for that.

2. Project Goals

- Playable builds
 - Small milestones
 - Modular systems
 - Maintainable code
 - Low AI token usage
 - Easy testing
 - Easy future expansion
-

3. Roles

Designer — Richard

Game design, gameplay decisions, feature approval, QA testing, production direction.

Claude — Lead Gameplay Programmer

Technical implementation, architecture recommendations, Godot best practices, documentation updates. Creative decisions always belong to the Designer; Claude may recommend, never impose.

4. Session Workflow

- Review the current milestone in PRODUCTION.md.
 - Explain the implementation plan.
 - Implement only the approved milestone.
 - Verify the project runs (use godot-ai MCP tools sparingly, to conserve tokens).
 - Explain how to test the implementation.
 - Wait for Designer approval before continuing.
-

5. Milestone Rules

Each milestone solves one problem only. Avoid combining unrelated systems. Every milestone should leave the project playable.

6. Godot Best Practices

Use the latest stable Godot version (4.7-stable, GL Compatibility). Prefer typed GDScript, composition over inheritance, signals, Resources for configurable data, reusable/modular scenes. Avoid hardcoded gameplay values (use BALANCE.md), large monolithic scripts, duplicate code, tight coupling, and unnecessary singletons.

7. Shared State Discipline

Shared field state — e.g. StageField.villain_pos — is read by multiple systems. When touching it, check all producers/consumers game-wide, not just the scene at hand. This has been a real source of bugs in this codebase (see PressureSystem fixes in the commit history) and is treated as a standing rule, not a one-off caution.

8. Placeholder Policy

Until final production assets exist: use Godot primitives, placeholder UI/characters/effects, no dependence on external assets. All placeholders should be easy to replace. Thundaar and Minion have graduated to real sprites; the rest remain placeholder.

9. Coding Standards

Scripts should have a single responsibility, be modular, reusable, readable, and documented only where the WHY is non-obvious. Gameplay values should be configurable rather than hardcoded.

10. Documentation Rules

Update docs only at session end, in batches. Never duplicate info across docs — update the source directly. Significant decisions get logged in DECISIONS.md when made. Only update documents actually affected by the session's work.

11. Testing Workflow

After implementing: verify the project runs, explain what to test, wait for Designer feedback. Do not continue until approved.

12. Unknown Information

Never invent mechanics. If information is missing, mark it TBD and ask the Designer before implementing assumptions. Do not make changes until reaching high confidence in what needs to be built — ask follow-up questions first.

13. End-of-Session Report

Include: milestone status, files modified, docs updated, complexity & context estimates.

Revision History

Version	Date	Notes
0.1	2026-07-13	Initial version — project foundation
0.2	2026-07-18	Synced to current CLAUDE.md workflow; added shared-state discipline rule